Google Googleyness & Leadership: STAR Answers

Source: Connect history (Viva Engage Infrastructure, Principal Engineer). Aim for 2–3 minutes per answer.

How to Answer

STAR	Time	Focus
Situation	~20s	Context, scale, why it mattered
Task	~15s	Your responsibility, what was at stake
Action	~90s	What **you** did (say "I"), trade-offs, how you brought people along
Result	~30s	Metrics, business impact, **what you learned**

What Google evaluates:

- **Emergent leadership:** steps up without authority, and also steps back to let others lead.
- **Intellectual humility:** admits mistakes, changes mind with data, credits others.
- **Comfort with ambiguity:** structures unclear problems, decides with incomplete data.
- **Bias to action with high standards:** proactive, but protects quality and the team.
- **Doing the right thing:** user and customer focus, psychological safety.

Rules: use one story per answer, keep numbers concrete, always end with a learning, avoid internal jargon (explain "HDB", "VENT", and "stamp" in one phrase), and don't badmouth anyone.

Story Bank

#	Story	Key numbers	Best for
S1	**PostgreSQL major-version upgrade** (stalled for years due to risk)	Zero production issues, 7 SLA clusters upgraded in parallel, replication lag eliminated, fewer incidents	Ambiguity, influence, high-stakes decision, speed vs quality
S2	**EUDB split routing / RouteKey** (took over a single-owner bottleneck service)	Delivered on time; fixed empty-tenant-ID data bug; made Teams Q&A EUDB-compliant	Ownership outside core, mistake (RouteKey incidents)
S3	**Elasticsearch to Kubernetes (AKS) + ES8**	Cluster creation days → hours; secret rotation weeks → minutes; 60% smaller footprint; unlocked semantic search	Process improvement, stakeholder, feedback
S4	**VENT lag ambiguity → Kafka deprecation design**	Root cause proven service-side; produced long-term eventing design	Disagreement, difficult stakeholder, ambiguity
S5	**Replica health-check gap in connection pooler**	Removed silent failure; faulty replicas now auto-removed	Improve without being asked, ambiguity
S6	**Mentoring: stalled service-testing workstream**	Unblocked cross-team testing for the upgrade	Mentoring, emergent leadership (stepping back)
S7	**Embeddings memory tuning**	66% latency reduction, 10% more user clicks	Customer impact, data-driven
S8	**Ubuntu upgrade setback + manager feedback**	Converted learnings into checklists and stage gates	Critical feedback, mistake
S9	**Ventsink incident: treated symptom, not cause**	~1 hour lost restarting services	Mistake, humility
S10	**On-call self-service: SAGE tool + AI Copilot demo**	Fewer infra requests, lower MTTR	Improve without being asked

> Fill in the **[F#]** markers using the Fill-in Sheet below before the interview.

Fill-in Sheet (confirm before interview)

Each item has a **draft** phrasing. Keep it if it's accurate, correct it if it isn't, or delete the claim if you can't stand behind it.

#	Question to answer	Draft phrasing (edit to match reality)
F1	How many service teams had to test the Postgres upgrade?	"about __ service teams across Viva Engage"
F2	Scale of the Postgres fleet	"__ clusters, ~__ TB, peak ~**__**K QPS, serving ~__**M users"
F3	Version jump and how long it had been deferred	"from Postgres __ to 16, deferred for ~__ years"
F4	Order of the late surprises during rollout (Q2)	"Two weeks before the SLA window, __; then __" (Azure core shortage, failed non-SLA upgrade from Azure API issues)
F5	Final decision on embeddings infrastructure (Q7)	"We __ (separate cluster / shared cluster with dedicated nodes), and I tuned memory, cutting latency 66%"
F6	Did the Ubuntu checklists and stage gates get reused? (Q9)	"Used for __ (PostgreSQL OS upgrade to 25.04 / HDB clusters)"
F7	Outage-time reduction from the parallel pipeline	"cut the total maintenance window from ~__ to ~__"
F8	Incident or MTTR improvement after the VENT, replica, or SAGE work	"replica-related read failures dropped from __ to __" or "MTTR down ~__%"

If you don't know a number, use a qualitative but true signal: "no repeat incidents of that class since," or "on-call stopped escalating it to me."

Sample: Ownership Outside Core Responsibilities

"Tell me about a time you took ownership of a project or problem outside your core responsibilities." Story: S2

- **S:** Our request-routing service (RouteKey) decides which data center ("stamp") each tenant's traffic goes to. Only one senior engineer knew it, so he had become a bottleneck for every project. EU Data Boundary (EUDB) compliance required tenants' data to be routed and stored in the EU, and Teams Q&A was blocked on it. My expertise was storage (Postgres, Elasticsearch), not routing.
- **T:** No one owned the routing changes. I volunteered to lead the design and delivery and to make sure we would never again depend on one person.
- **A:**
  - Learned the codebase and designed split routing, so new EU tenants route to the EU stamp while existing tenants keep working.
  - Ran recurring syncs with partner teams and tracked the dependencies between teams explicitly so nothing slipped.
  - While digging in, found a long-standing data bug (empty tenant IDs in the routing DB) that caused hard-to-detect customer issues, and shipped a fix that prevents recurrence.
  - Paired with a teammate on API design, code reviews, and test cases so he could own changes independently.
  - Joined the EUDB data-migration design to define how the relocation service communicates across stamps.
- **R:** Delivered within the estimated timeline. Teams Q&A became EUDB-compliant for new customers. We went from one engineer who could support routing to several, and my manager called out that we were "in a much better position as a team." *Learning:* we did have early incidents from rushing changes for service teams (see Q5), which taught me to slow down on critical-path infrastructure.

🌀 Navigating Ambiguity & Decision Making

Q1. High-stakes decision with incomplete or conflicting data

Story: S1

- **S:** Our self-hosted PostgreSQL fleet backs every Viva Engage service. It was several major versions behind, which blocked security patches (open CVEs) and OS upgrades. The upgrade had been deferred for years because a failure could take down the product, and the options were contested: in-place upgrade, logical replication, or a managed Azure service.
- **T:** Pick an upgrade path that minimized downtime and risk, and convince senior principals to back it.
- **A:**
    - Wrote an options document comparing downtime, rollback ability, operational risk, and team effort for each approach.
    - Where data was missing, I generated it. I ran a **proof of concept on the production database (HDB)** without disrupting live traffic, so the decision rested on real behavior, not opinions.
    - Defined explicit rollback criteria up front, which made the decision reversible and lowered the stakes.
    - Presented it to three senior principals who had differing opinions, addressed each concern with POC data, and got alignment.
- **R:** The chosen approach shipped across all clusters with **zero production issues**, eliminated replication lag, reduced incidents, improved indexing performance, and unblocked security and OS upgrades. *Learning:* the POC missed a cross-region replication issue, so I now validate cross-region and cross-environment dependencies first in any POC.

Q2. Requirements changed drastically at the last minute

Story: S1 rollout. Confirm the sequence of events in **[F4]**.

- **S:** During the production upgrade rollout, we had a sequence of changes planned across SLA (customer-facing) and non-SLA clusters. Late in the process we hit two surprises: an **Azure core-capacity shortage** in our region, and a non-SLA upgrade that failed because of Azure API issues.
- **T:** Keep the rollout safe and on schedule without burning the team or surprising leadership.
- **A:**
    - Paused and re-sequenced: protected the SLA cluster windows first and moved non-SLA work behind them.
    - Worked with Azure capacity contacts to secure cores and adjusted the batch sizes of the parallel automation pipeline to fit the available capacity.
    - Reset expectations with service teams on new windows, and clarified each team's role in validation (there had been confusion).
    - Gave leadership a mitigation plan and a new timeline. I'll admit my first update was less clear than it should have been.
- **R:** All 7 SLA clusters were upgraded with no production issues. *Learning:* I now track capacity proactively before rollouts and send stakeholders updates on a regular cadence, not only when something breaks.

Q3. Complex problem with no clear solution

Story: S5 (alternate: S4)

- **S:** Engage services saw intermittent failing read queries. Postgres looked healthy, the app looked healthy, and the logs pointed nowhere specific.
- **T:** I was leading the investigation to find the root cause, not just mitigate.
- **A:**
    - Broke the read path into layers: app → connection pooler → replicas. I formed a hypothesis for each layer and tested them one by one.
    - Found one replica that was unresponsive because of a disk problem. The pooler's health check didn't check disk availability, so it kept routing reads to that replica.
    - Fixed the health check and established a protocol to **automatically pull faulty replicas** out of rotation.

- **R:** Removed a silent failure mode, improved read availability, and prevented a whole class of future incidents. *Learning:* when it's unclear what's wrong, map the system, list hypotheses, and eliminate them with data before acting.

🚀 Emergent Leadership & Influencing

Q4. Influenced without formal authority

Story: S1 (cross-team rollout)

- **S:** The Postgres upgrade required every service team in Viva Engage to test against the new version. None of them reported to me, and they all had their own roadmaps.
- **T:** Get **[F1]** service teams to test, sign off, and accept upgrade windows.
- **A:**
    - Went through engineering managers first to explain the "why" (security CVEs, reliability) and got explicit buy-in that it was a dependency.
    - Lowered their cost: stood up a LAB environment with the new version, made our infrastructure tools compatible, and built an **automation pipeline** that shortened their outage windows by upgrading clusters in parallel.
    - Made progress visible with a shared test-tracking sheet and regular check-ins, so peer visibility did the nudging.
- **R:** Every team validated, and we upgraded 7 SLA clusters with zero production issues. *Learning:* influence comes from making the right thing the easy thing and making progress visible.

Q5. Advocated a change that faced heavy resistance

Story: Pushing back on rushed infrastructure changes (from S2 and on-call)

- **S:** Service teams were used to infrastructure making quick changes on request, and to running their own heavy queries or changing network settings directly on servers. After a few routing incidents caused by rushed changes, I believed we needed a stricter bar.
- **T:** Shift the culture toward validation before production, even though teams expected the old speed.
- **A:**
    - Owned my part publicly: the incidents came partly from changes I pushed too fast.
    - Proposed a clear process: test plans, staged rollouts, and no direct production modifications by service teams. I explained the customer impact in their terms.
    - Got my manager's explicit support, then handled pushback case by case, offering safe alternatives (for example, query optimization help or read replicas) instead of a flat "no."
- **R:** Fewer infrastructure-caused incidents. Teams started coming to us with their expensive top queries to optimize instead of running them live. *Learning:* resistance drops when you pair a "no" with a better "yes."

Q6. Supported a struggling peer or junior teammate

Story: S6 (mentoring; also shows stepping back)

- **S:** For the Postgres upgrade, the service-team testing workstream had stalled. Teams were unclear on what to test, and a junior teammate on it was overwhelmed.
- **T:** Unblock testing **without** taking it over myself. I wanted her to own it.
- **A:**
    - Coached her one-on-one on structuring it: a test-case tracker, owners, and dates.
    - Helped her draft communications to service teams, then let her send them and run the syncs.
    - Stayed behind the scenes for escalations only, and gave her credit visibly.
- **R:** Testing got back on track with clarity across all teams. My manager received positive feedback from her about the coaching. Similarly, I ramped another teammate up on routing and Elasticsearch until he could handle incidents independently. *Learning:* my leverage as a senior engineer is growing owners, not doing the work myself. Earlier feedback that I sometimes "covered for others" pushed me here.

🤝 Intellectual Humility & Team Dynamics

Q7. Strongly disagreed with a manager or teammate

Story: S3 (embeddings on Elasticsearch)

- **S:** The search team needed infrastructure for AI embeddings. I pushed to implement it on our existing Elasticsearch infrastructure. My manager wasn't convinced and wanted us to consider a separate cluster.
- **T:** Reach the right decision, not win the argument.
- **A:**
    - Laid out my reasoning: reuse, speed, and lower operational cost.
    - Listened to his concern, which was the operational blast radius, isolation, and scaling of mixing workloads. I realized I had optimized for implementation speed rather than long-term operability.
    - Re-evaluated using criteria we both agreed on (isolation, cost, on-call burden, and scaling), which led to scaling the Elasticsearch infrastructure with dedicated hardware for embeddings. **[F5]**
- **R:** Embeddings performance and capacity improved, and later memory tuning cut latency by 66% and increased user clicks by 10%. *Learning:* in a disagreement, I first agree on the decision criteria. That also taught me to voice my ideas strongly while considering every angle.

Q8. Wrong about something or made a mistake

Story: S9 (alternate: S2's RouteKey incidents)

- **S:** During an on-call incident, a service saw high latency and 5xx errors writing to our event pipeline (ventsink/Kafka).
- **T:** Mitigate quickly.
- **A (mistake):** I spent nearly an hour restarting ventsink and Kafka because I assumed the components were at fault. I didn't stop to ask why **only the East region** was affected, which pointed to cross-region network connectivity.
- **A (adapting):** Once I saw it, I pivoted to the network path and mitigated it. I then wrote it up in the postmortem, and updated our runbooks and incident guides with a "scope first" step (which regions, which components, and what changed).
- **R:** Mitigation time (MTTR) on later incidents improved. *Learning:* before acting, I determine the blast radius; a symptom limited to one region rarely has a component-level cause.

Q9. Received difficult or critical feedback

Story: S8

- **S:** After the Ubuntu OS upgrades on our database clusters, I reflected that I had "missed early signals." My manager told me my reflection was **too high-level** and didn't translate into specific, repeatable changes.
- **T:** Turn the lesson into mechanisms, not intentions.
- **A:**
    - My first reaction was some defensiveness, because I *had* reflected. Then I recognized he was right: an intention isn't a process.
    - Built a **pre-flight validation checklist**, explicit **rollout stage gates** (LAB → STAGE → non-SLA → SLA), defined success and failure criteria for each stage, and **automated alerts** for early signals tied to metrics.
    - Documented upgrade procedures and set a goal of training two teammates to run upgrades without me.
- **R:** The checklists and stage gates became the template for the next OS and Postgres upgrades. **[F6]** *Learning:* every lesson I learn now has to become a checklist, a gate, or an alert.

Q10. Difficult cross-functional stakeholder with different priorities

Story: S4 (alternate: escalations from the search team)

- **S:** A partner team saw lag in VENT, our event pipeline, and insisted it was an infrastructure problem. Escalations were going back and forth, and both teams had different priorities.
- **T:** Resolve the ambiguity and get the issue fixed at the right layer, without damaging the relationship.
- **A:**
    - Proposed an investigation with shared metrics instead of debating opinions. I showed that the platform delivered events on time and the lag was in **service-side consumption**.
    - Walked their engineers through the data and taught them how to read the dashboards themselves.
    - Offered to help: I suggested consumer-side fixes and also produced a **VENT/Kafka improvement and deprecation design**, so the platform itself would become more observable and better alerted.
- **R:** Got alignment on the root cause and ownership. The fix happened at the right layer, and we now have a long-term eventing roadmap. *Learning:* with shared data, a "your problem vs. my problem" dispute becomes a joint problem.

💡 Execution & High Standards

Q11. Improved a process or system without being asked

Story: S3 (alternate: S10)

- **S:** Creating an Elasticsearch cluster took **several days** of manual work, and rotating secrets and certificates took **weeks**. It was a major source of toil and security risk for the on-call team.
- **T:** No one had assigned this. I saw it as a recurring tax on the team.
- **A:**
    - Proposed and delivered a move of Elasticsearch to Kubernetes (AKS), a service managed by Azure.
    - Automated cluster creation and secret rotation into one-click workflows, and removed the manual commands.
    - Upgraded to ES8 along the way.
    - Ran one-on-ones with teammates to find gaps in the runbooks and fixed them.
- **R:** Cluster creation went from **days to hours**, secret rotation from **weeks to minutes**, and the footprint is **60% smaller**. ES8 enabled **semantic search** for Viva Answers. *Learning:* feedback showed we caused some Sev-1 alerts during the early scale-up, so I now load-test and stage the rollout before handing a new system to on-call.

Q12. Speed versus quality under a tight deadline

Story: S1 (with the RouteKey contrast)

- **S:** Service teams wanted the Postgres upgrade done fast, and leadership worried about the timeline, especially for rollback testing.
- **T:** Hit the timeline without compromising a system where any error is customer-visible.
- **A:**
    - Separated what was fixed from what was flexible. Quality gates were fixed: rollback tested, LAB → STAGE → production, and service sign-off. The process was flexible.
    - Got speed through automation: a pipeline that upgraded 7 SLA clusters in parallel, which cut total outage time.
    - Was explicit with stakeholders that rollback testing would take longer than expected, and why.
- **R:** Delivered with **zero production issues**. *Contrast I share:* earlier, on RouteKey, I prioritized speed and caused incidents. My manager reinforced that our team "cannot trade testing for speed." *Learning:* the right way to go faster is to automate, never to cut validation, on critical infrastructure.

Q13. Prioritizing multiple high-priority tasks and saying "no"

Story: Competing priorities (security, Postgres, COSMIC, EUDB, and on-call)

- **S:** In one period, I was the go-to expert for Postgres and Elasticsearch while also working on security and CVE remediation (the top org priority), the Postgres upgrade, onboarding services to the COSMIC platform for government cloud (GCC), EUDB, and on-call. Direct requests from the search team were getting escalated to my manager.

- **T:** Deliver the critical work without becoming the bottleneck.
- **A:**
    - Ranked work with my manager: security first (non-negotiable), then compliance-driven upgrades, then everything else.
    - Said "not now" to direct requests by **coaching requesters onto the team's intake process** instead of answering them ad hoc, which also reduced escalations.
    - Delegated through mentoring: others ran upgrades, MSI-status reporting (managed identity, replacing stored credentials), and routing changes.
    - Introduced sprint estimation and planning discipline, because work was regularly carrying over between sprints.
- **R:** Shipped the Postgres upgrade, EUDB routing, and ES modernization. Fewer requests depended on me as the expert, and escalations dropped. *Learning:* a principal engineer's "no" should come with a path (a process, a person, or a date).

🎙️ 2-Minute Spoken Scripts (Top 5 Stories)

Read these aloud and time yourself. Each is about 250–300 words (~2 minutes). Use your own words in the interview; these are a rehearsal base, not something to memorize.

Script A: Postgres upgrade (ambiguity, influence, speed vs quality)

> "Our self-hosted PostgreSQL fleet backs every Viva Engage service, **[F2]**. It was several major versions behind, **[F3]**, which blocked security patches and OS upgrades. It kept getting deferred because a failed upgrade could take the whole product down.
>
> I took ownership of finding a safe path. There were competing options and senior principals disagreed, so instead of arguing I wrote an options document comparing downtime, rollback, and operational risk. Then I ran a proof of concept on our production database without affecting live traffic. That data is what got three senior principals aligned.
>
> The harder part was people. **[F1]** service teams had to test, and none reported to me. I went through their managers to explain the security and reliability case, stood up a LAB environment, and coached a junior teammate to run the testing workstream with a shared tracker.
>
> Leadership was worried about timelines, so I got speed from automation, not shortcuts. I built a pipeline that upgraded 7 SLA clusters in parallel, **[F7]**, and kept the rollback-testing gate even though it took longer.
>
> We shipped with zero production issues, eliminated replication lag, and reduced incidents. What I'd do differently: the POC missed a cross-region replication issue, and a late Azure core shortage surprised us. I now validate cross-region dependencies first and track capacity as an explicit risk."

Script B: EUDB / RouteKey (ownership outside core)

> "Our routing service decides which data center each tenant's traffic goes to. Only one senior engineer understood it, so every project waited on him. EU Data Boundary compliance needed EU tenants routed and stored in the EU, and Teams Q&A was blocked. My area was storage, not routing, but no one owned it, so I volunteered.
>
> I learned the codebase and designed split routing so new EU tenants go to the EU stamp while existing tenants keep working. I ran regular syncs with partner teams and tracked dependencies between teams explicitly. While digging in, I found a long-standing bug: empty tenant IDs in the routing database were causing hard-to-trace customer issues. I fixed it at the source.

I deliberately didn't want to become the new bottleneck, so I paired with a teammate on the API design, code reviews, and test cases until he could make changes and handle incidents himself.

We delivered on schedule and Teams Q&A became EUDB-compliant for new customers. We went from one person who could support routing to several.

The honest part: early on, service teams pushed for quick changes and I shipped some too fast, which caused incidents. I learned that on critical infrastructure I have to hold the testing bar and push back, even when others expect speed."

Script C: Elasticsearch on AKS (improving without being asked)

"Running Elasticsearch was pure toil. Creating a cluster took several days of manual work, and rotating secrets and certificates took weeks, which was also a security risk. No one asked me to fix it, but it was a recurring tax on on-call.

I proposed moving Elasticsearch onto Azure Kubernetes Service and upgrading to ES8. I automated cluster creation and secret rotation into one-click workflows and removed the manual commands. Viva Answers had a deadline that depended on ES8, so I joined their standups so the plan and timeline were clear to them.

Cluster creation went from days to hours, rotation from weeks to minutes, and the footprint shrank by 60%. ES8 unlocked semantic search for Viva Answers.

Two lessons. My manager pointed out that I hadn't held the search team accountable for ES8 testing readiness early on, and that cost about four weeks. And our early scale-up caused some Sev-1 alerts. Now I agree on readiness criteria with partner teams up front, load-test before go-live, and run 1:1s with teammates to close runbook gaps, so on-call isn't the one finding the problems."

Script D: VENT lag (difficult stakeholder, disagreement)

"A partner team saw lag in VENT, our event pipeline, and insisted it was an infrastructure problem. It escalated back and forth, and both teams had other priorities.

Rather than debating, I proposed we look at the same data. I traced the event path end to end and showed that the platform delivered events on time and the lag built up in how the service consumed them. I walked their engineers through the dashboards so they could see it themselves, not just take my word for it.

I didn't stop at 'it's your problem.' I suggested consumer-side fixes, and I acknowledged that our platform was hard to observe, so I wrote a VENT improvement and Kafka deprecation design to make eventing more reliable and better alerted.

We aligned on the root cause, it got fixed at the right layer, and we now have a long-term eventing roadmap. What I took away: shared data turns a blame conversation into a joint problem, and there's almost always something my side can improve too."

Script E: Ventsink incident (mistake, humility)

"During an on-call incident, a service was seeing high latency and 5xx errors writing to our event pipeline. I assumed the components were at fault and spent almost an hour restarting ventsink and Kafka.

What I missed was that only the East region was affected, which pointed to cross-region network connectivity, not the services. Once I stepped back and looked at the scope, I moved to the network path and mitigated it.

> I owned that openly in the postmortem. Then I changed how we work: I added a 'scope first' step to our runbooks and incident guides (which regions, which components, what changed), and later built SAGE, a self-service debugging tool for on-call, so engineers can see the blast radius quickly.
>
> The lesson I carry: determine the blast radius before acting. An issue limited to one region rarely has a component-level cause."

🔮 Hypothetical / Situational Questions

Google often asks "What would you do if…" questions. Answer with a **framework**, then **anchor it to a real story** ("I actually faced something similar when…"). This combines judgment with evidence, and it's the strongest answer pattern.

Framework (CLEAR):

1. **C**larify the goal and constraints, and ask questions before acting.
2. **L**isten to the stakeholders and understand their incentives.
3. **E**valuate options using explicit criteria (user impact, risk, reversibility, cost).
4. **A**ct: decide, communicate, timebox, and make it reversible where possible.
5. **R**eflect: measure the outcome and share the learnings.

H1. "Your team is about to launch, and you discover a bug that could affect a small percentage of users. The PM wants to ship. What do you do?"

- Clarify how severe the bug is: data loss or security issue versus a cosmetic problem. Find out how many users, whether it's reversible, and whether a feature flag or rollback exists.
- If it affects user data or security, I wouldn't ship. I'd propose a path instead, such as a staged rollout excluding the affected users, a fast fix, or a new date.
- If it's minor, ship behind a flag with monitoring, a documented known issue, and a fix date.
- Escalate with data, not opinion, then disagree and commit if the decision goes the other way.
- **Anchor:** "On our Postgres upgrade, service teams wanted speed, but I held the rollback-testing gate and delivered with zero production issues. Earlier, on RouteKey, I shipped too fast and caused incidents, which is why I hold this line now."

H2. "Two senior engineers on your team strongly disagree on a technical design, and it's blocking progress. You're not their manager."

- Meet each one separately to understand their concerns. Usually they're optimizing for different things.
- Bring them together to agree on **decision criteria** first (latency, operability, cost, timeline), then score the options against them.
- If it's still tied, propose a timeboxed POC or benchmark to settle it with data.
- Document the decision and the alternatives considered, so the person who "lost" feels heard.
- **Anchor:** "When three senior principals had different views on the Postgres upgrade path, I ran a production POC and wrote an options document. That data aligned everyone."

H3. "You join a new team and find their on-call is burning people out, but no one has asked you to fix it."

- Listen first: shadow on-call, read the last few months of incidents, and talk to engineers one-on-one.
- Quantify the problem: pages per week, top alert sources, and toil hours.
- Pick one high-leverage fix, deliver it, and show the impact. Then propose a broader plan to the manager.
- Bring others along as owners rather than doing it all myself.
- **Anchor:** "On Elasticsearch, secret rotation took weeks and cluster creation took days. I moved it to AKS and automated both, and I built the SAGE tool so on-call could self-serve debugging."

H4. "Leadership changes the priority mid-project and asks your team to drop current work. The team is frustrated."

- Clarify the *why* with leadership so I can explain it credibly.
- Acknowledge the team's frustration openly, because psychological safety matters.
- Wind the old work down cleanly: document its state so it can be resumed, and avoid leaving half-finished changes in production.
- Re-plan the new work with the team, and give people ownership of pieces they care about.
- **Anchor:** "When security and CVE work became the org's top priority, I re-ranked my work with my manager, wound down lower-priority items, and coached requesters onto our process."

H5. "A teammate keeps missing deadlines, and it's starting to affect your project."

- Talk privately and with curiosity: "What's getting in the way?" There may be ambiguity, overload, or a personal issue.
- Break the work into smaller milestones with clear expectations, and offer pairing.
- If it continues, involve the manager transparently. I'd tell the teammate I'm doing it.
- **Anchor:** "A teammate was struggling with the stalled service-testing workstream. I coached her on structure and communication, and she got it back on track and owned it."

H6. "You're asked to estimate a project with many unknowns, and leadership wants a firm date."

- Give a range with explicit assumptions, not a single date.
- Identify the riskiest unknowns and propose a short spike or POC to reduce them, then commit to a firmer date after it.
- Set checkpoints where the estimate is revisited, and communicate early if it slips.
- **Anchor:** "For the Postgres upgrade, I ran a POC before committing. A last-minute Azure core shortage taught me to track capacity as an explicit risk."

H7. "A partner team blames your system for an outage, but you believe the cause is on their side."

- Focus on restoring service first, and assign blame later.
- Propose a joint, blameless investigation using shared metrics and timelines.
- If it's their issue, share the data privately and respectfully, and offer help to fix it.
- Look for what *my* system could do better anyway: alerting, guardrails, documentation.
- **Anchor:** "With VENT lag, I used metrics to show the cause was on the consumer side. I helped them fix it and also wrote a design to make VENT more observable."

H8. "You realize a decision you advocated for months ago was wrong, and the team has built on it."

- Raise it quickly and own it publicly. Sunk cost isn't a reason to continue.
- Quantify the cost of changing course now versus later, and present the options.
- Propose an incremental migration path rather than a big rewrite.
- **Anchor:** "On embeddings, my manager challenged my approach. I re-evaluated against operability and isolation, and we scaled with dedicated hardware."

H9. "You have the skills to fix a problem in another team's codebase much faster than they would. Do you?"

- Ask before acting: it's their system, their on-call, and their context.
- Offer to pair or to send a PR they review, so they keep ownership and learn.
- In an urgent customer-impacting case, coordinate on mitigating first, then hand back.
- **Anchor:** "I designed and code-reviewed RouteKey changes with a teammate instead of doing them myself, which removed a single point of failure."

Tips for hypotheticals

- Don't jump to a solution. Spend the first 20 seconds on clarifying questions or stated assumptions.
- Name the trade-offs explicitly (user impact versus speed, short term versus long term).
- Always close with "and in fact, I faced something similar when…" to tie in a real story.

Quick Reference: Question → Story

Question	Primary	Backup
Ownership outside core	S2	S3
Decision with incomplete data	S1	S3
Last-minute change	S1 (capacity / Azure API)	S3 (ES8 timeline)
Unclear problem	S5	S4
Influence without authority	S1	S2
Change facing resistance	Push back on rushed changes	S3
Mentoring a struggling peer	S6	Ramping a teammate on routing / ES
Disagreement	S7 embeddings	S4
Mistake	S9 ventsink	S2 RouteKey incidents
Critical feedback	S8 Ubuntu	"Covering for others" (S3)
Difficult stakeholder	S4 VENT	Search team escalations
Improve without being asked	S3 ES on AKS	S5, S10
Speed vs quality	S1 vs RouteKey	S3
Prioritize / say no	Competing priorities	Security-first ranking

Questions to Expect as Follow-ups

- "What would you do differently?" → Use the *Learning* line.
- "What did others do vs. you?" → Credit teammates, then state your specific actions.
- "How did you measure success?" → Cite the numbers. If none, name the signal (incidents, MTTR, escalations).
- "What if they had still said no?" → Escalate with data, timebox, and disagree and commit.